
MÉTRICAS DE DESEMPENHO

Computação Paralela · Unidade 1

Agosto, 2026

ROTEIRO

- 1 | Tempo de execução e fator de aceleração (*speed-up*)
- 2 | Eficiência e o *sweet spot*
- 3 | Escalabilidade: Leis de Amdahl e Gustafson
- 4 | Escalabilidade forte e fraca
- 5 | Estudo de caso: aplicação MPI em um cluster

POR QUE MEDIR?

Paralelizar um programa é "fácil", ganhar o desempenho é difícil! Saber se valeu a pena exige **medir**.

O CÓDIGO ESTÁ CERTO?

Resultado correto não diz nada sobre desempenho: um programa paralelo pode ser mais *lento* que o sequencial.

QUANTOS RECURSOS USAR?

Em um cluster compartilhado, pedir 256 cores para um programa que só aproveita 32 é desperdício: o seu e o dos outros.

ONDE OTIMIZAR?

As métricas apontam *qual* gargalo bateu: fração sequencial, comunicação, banda de memória ou desbalanceamento.

PONTO DE PARTIDA: O TEMPO DE EXECUÇÃO

NOTAÇÃO

- $T_1(w)$: tempo do *melhor* algoritmo sequencial para o trabalho w
- $T_p(w)$: tempo com p unidades ativas
- p : número de unidades ativas (cores, threads, processos)
- w : carga de trabalho (tamanho do problema)

Cuidado: T_1 é o melhor algoritmo *sequencial* conhecido, e não o programa paralelo rodando com $p = 1$.

Rodar a versão paralela com uma unidade ativa carrega todo o custo da paralelização (criação de threads, buffers, sincronização) sem nenhum benefício. Usar esse tempo como referência **infla artificialmente** todas as métricas.

O tempo é a métrica que interessa ao usuário. As demais métricas existem para **explicar** o tempo e prever como ele muda com mais recursos.

ACELERAÇÃO E EFICIÊNCIA

As duas métricas básicas de avaliação

FATOR DE ACELERAÇÃO (SPEED-UP)

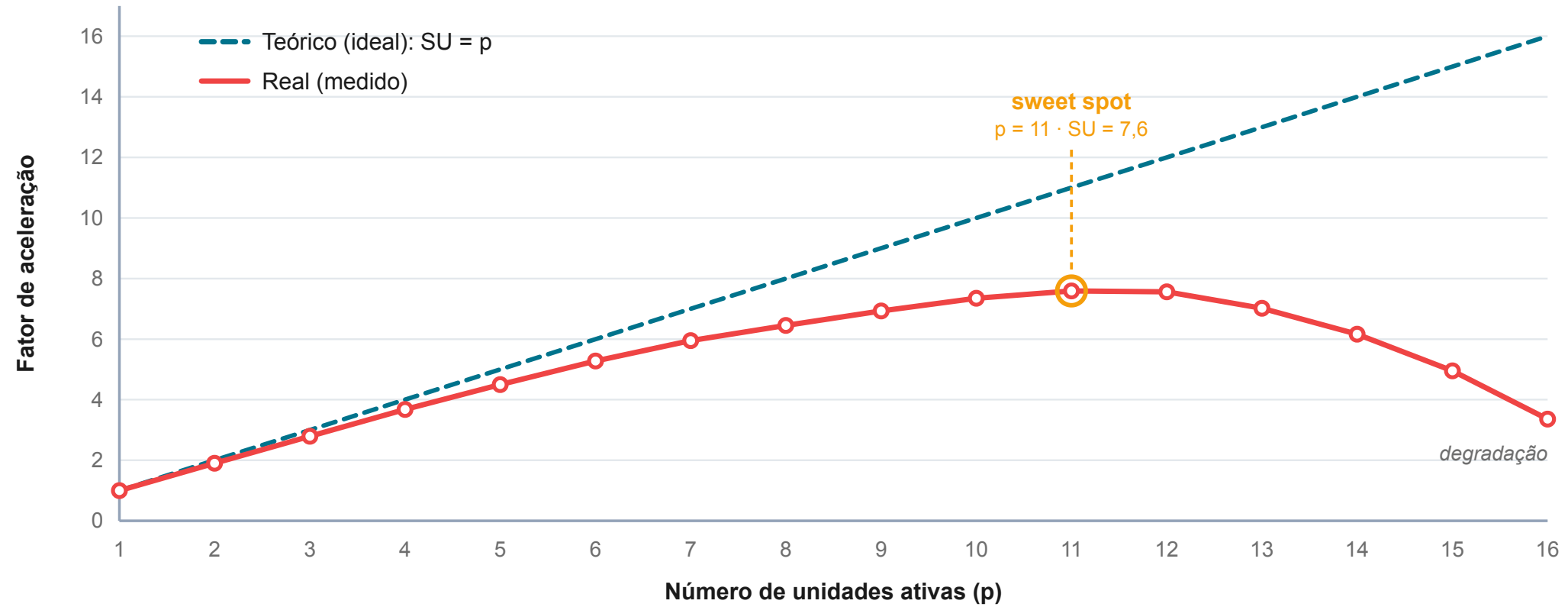
Indica **quantas vezes** a versão paralela ficou mais rápida que a sequencial.

$$SU_p(w) = \frac{T_1(w)}{T_p(w)}$$

O valor só significa alguma coisa **comparado a p**: é *p* que define o ganho perfeito.

Faixa	Classificação	Significado
$SU < 1$	retardo	A versão paralela ficou <i>mais lenta</i> que a sequencial
$1 \leq SU < p$	sublinear	O caso real: parte do ganho é consumida pelos complicadores
$SU = p$	linear (ideal)	Cada unidade ativa contribui integralmente
$SU > p$	superlinear	Raro: em geral efeito de cache; suspeite de erro de medição

SPEED-UP: IDEAL VS. REAL



ANÁLISE DO SPEED-UP

- Cada aplicação tem **a sua curva**, mais ou menos acentuada, dependendo da carga de trabalho e da facilidade de quebrá-la em subtarefas independentes
- Toda aplicação tem um número de unidades ativas **ideal**, o *sweet spot*, que não é necessariamente o maior p disponível
- Depois do *sweet spot*, acrescentar recursos **piora** o tempo: o custo de coordenar as unidades supera o ganho de dividir o trabalho

Um programa paralelo pode ficar **mais lento** quanto mais recursos recebe. Mais unidades ativas nunca é garantia de menos tempo.

EFICIÊNCIA

Indica a **taxa de utilização média** das unidades ativas, ou seja, se os recursos foram bem aproveitados.

$$E_p(w) = \frac{SU_p(w)}{p}$$

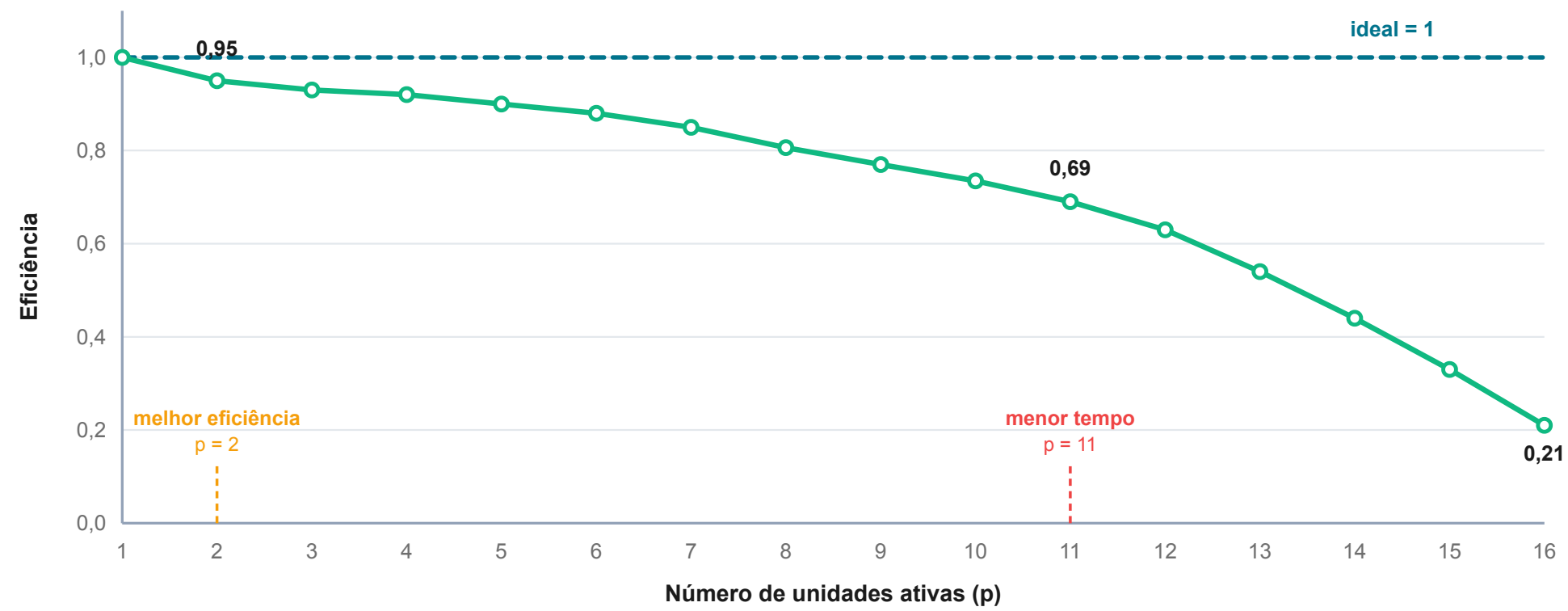
COMO LER

- **E = 1**: cada unidade ficou 100% do tempo útil (ideal)
- **E = 0,5**: metade do potencial da máquina foi desperdiçada

O QUE ELA MEDE

A distância entre o medido e o ideal. Normalmente as unidades passam parte do tempo **esperando** resultados de vizinhos, e isso derruba a taxa de utilização.

EFICIÊNCIA: A MESMA EXECUÇÃO, OUTRA PERGUNTA



Mesmos dados do gráfico anterior, agora divididos por p .

A MELHOR EFICIÊNCIA NÃO DÁ O MENOR TEMPO

p	1	2	4	8	11	16
Speed-up	1,00	1,90	3,68	6,45	7,59	3,36
Eficiência	1,00	0,95	0,92	0,81	0,69	0,21

MENOR TEMPO: P = 11

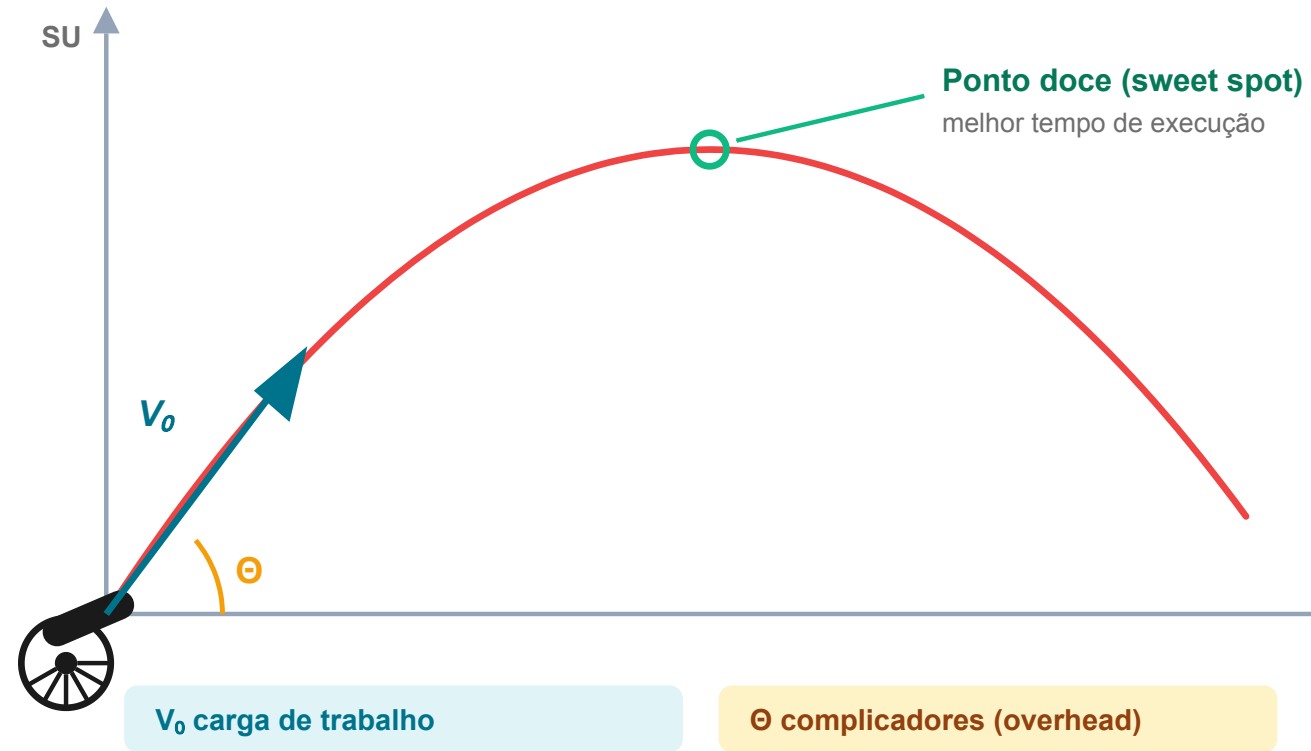
Eficiência de apenas 0,69: quase um terço da máquina desperdiçado.

MELHOR EFICIÊNCIA: P = 2

Aproveitamento de 0,95, mas o tempo é quatro vezes maior que em p = 11.

Não existe resposta única: o *p* certo depende do que você está otimizando: **tempo até o resultado** ou **custo computacional**.

FATORES QUE IMPACTAM O SPEED-UP



Mais carga de trabalho empurra o *sweet spot* para a direita; mais complicadores o puxam para a esquerda.

OS COMPLICADORES

Todo trabalho que a versão sequencial **não precisa fazer**:

COMUNICAÇÃO

Troca de dados entre processos: latência da rede e volume transferido. Cresce com p .

SINCRONIZAÇÃO

Barreiras, seções críticas e reduções. O processo mais lento dita o ritmo de todos.

DESBALANCEAMENTO

Divisão desigual do trabalho: unidades ociosas esperando as sobrecarregadas.

FRAÇÃO SEQUENCIAL

Trechos que não paralelizam: leitura de entrada, escrita de saída, inicialização.

CONTENÇÃO DE MEMÓRIA

Cores do mesmo nó disputam a banda de memória e os níveis de cache compartilhados.

TRABALHO EXTRA

Cálculos replicados, cópias de *buffers*, estruturas duplicadas por processo.

ESCALABILIDADE

Como o desempenho evolui quando acrescentamos recursos

DEFINIÇÃO

Escalabilidade é a capacidade de um sistema de usar **mais recursos** (processadores, cores, nós) para **reduzir o tempo de execução** mantendo a eficiência, idealmente igual a 1.

HARDWARE

Capacidade da máquina de entregar mais poder computacional quando se acrescentam componentes: mais cores, mais nós, rede mais rápida.

SOFTWARE

Capacidade do programa de *converter* esse poder adicional em tempo menor. É a razão entre o speed-up obtido e o speed-up ideal.

Escalabilidade é uma propriedade do **par aplicação + máquina**. O mesmo código escala de formas diferentes em plataformas diferentes.

LEI DE AMDAHL (1967)

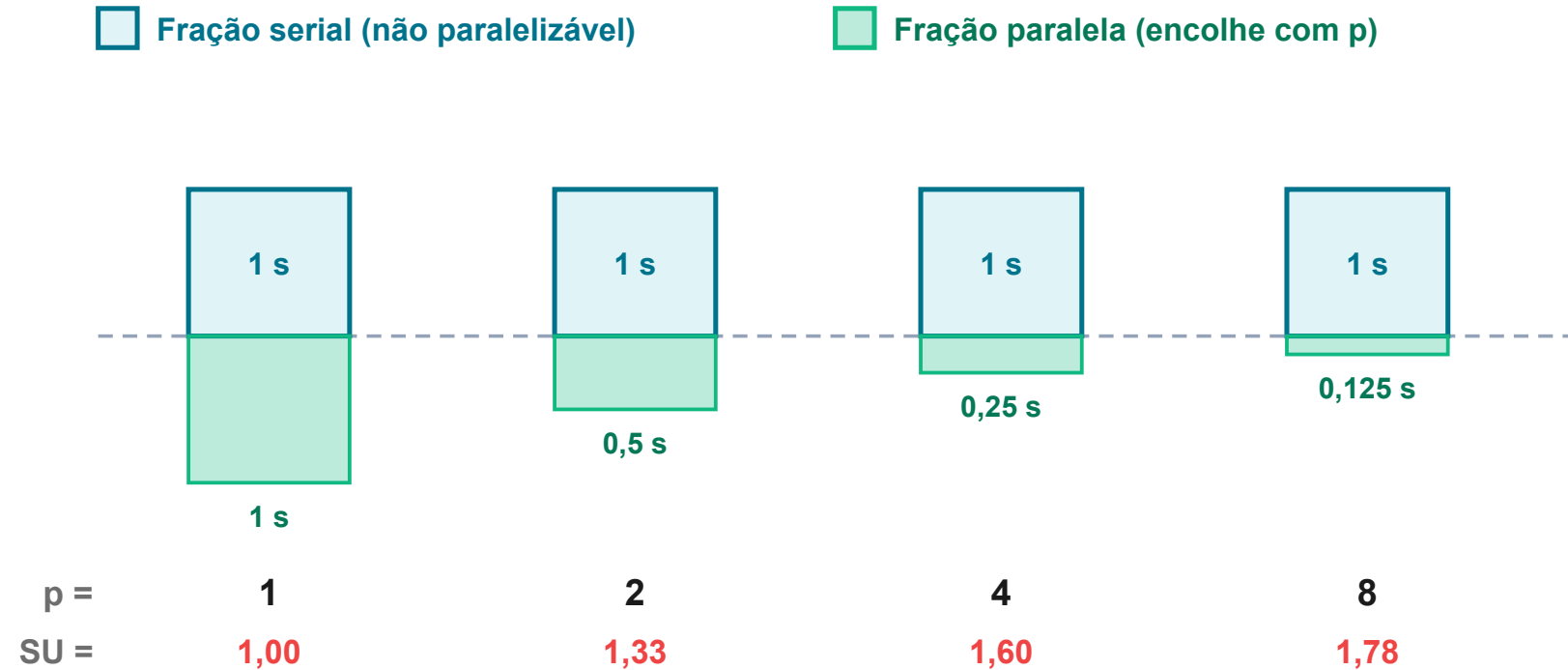
O ganho ao paralelizar uma tarefa de **tamanho fixo** é limitado pela fração que **não pode** ser paralelizada.

$$SU(N) = \frac{1}{s + p/N}$$

s = fração sequencial · p = fração paralelizável ($s + p = 1$) · N = número de processadores

Exemplo: se 90% do programa paralelizam ($s = 0,1$), com 10 processadores o ganho é de apenas 5,26×: $1 / (0,1 + 0,9/10) = 5,26$

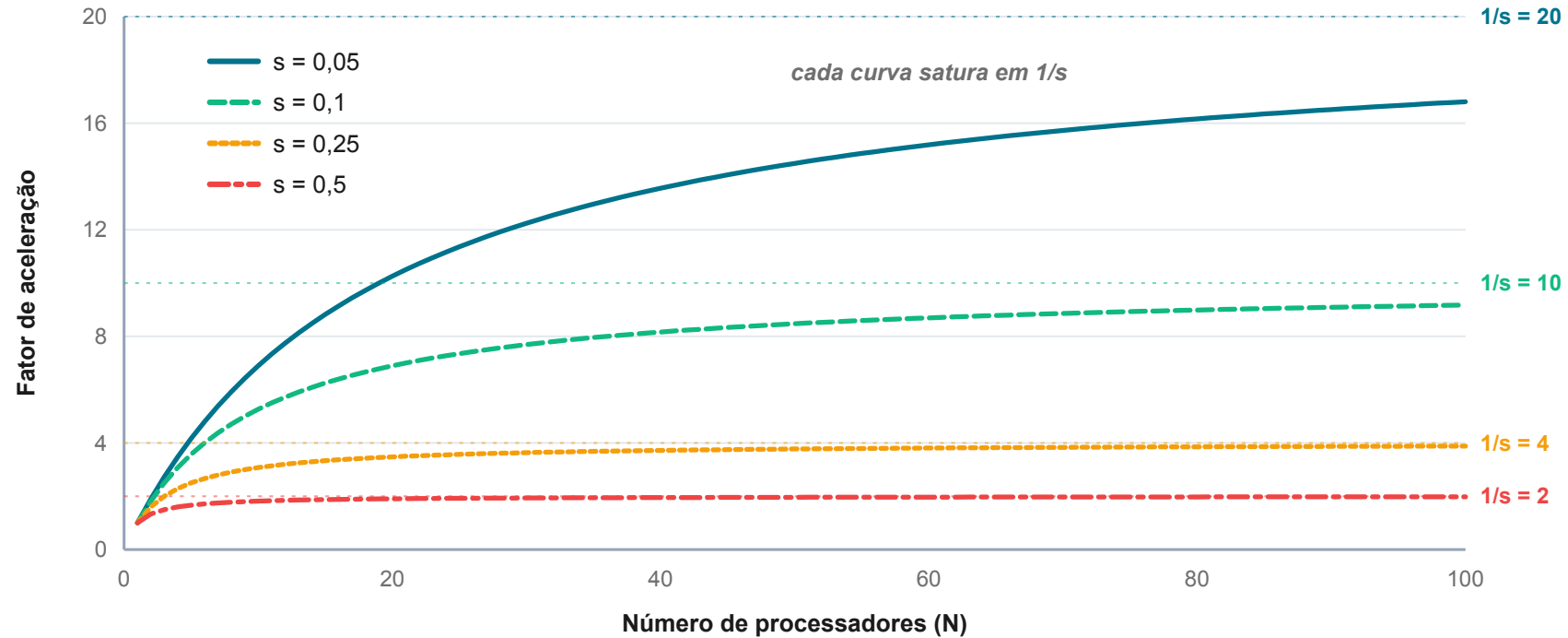
VISUALIZANDO A LEI DE AMDAHL



Problema de tamanho fixo: só a parte paralela encolhe.

$$SU(8) = 2 / 1,125 = 1,78 \cdot \text{Eficiência} = 0,22$$

O TETO IMPOSTO PELA FRAÇÃO SEQUENCIAL



Com $N \rightarrow \infty$, o speed-up tende a $1/s$. Com 5% de código sequencial, o limite é 20 $\times$, mesmo com um milhão de processadores.

LEI DE GUSTAFSON (1988)

Se o **tamanho do problema pode crescer** junto com os recursos, a fração sequencial fica cada vez menos representativa.

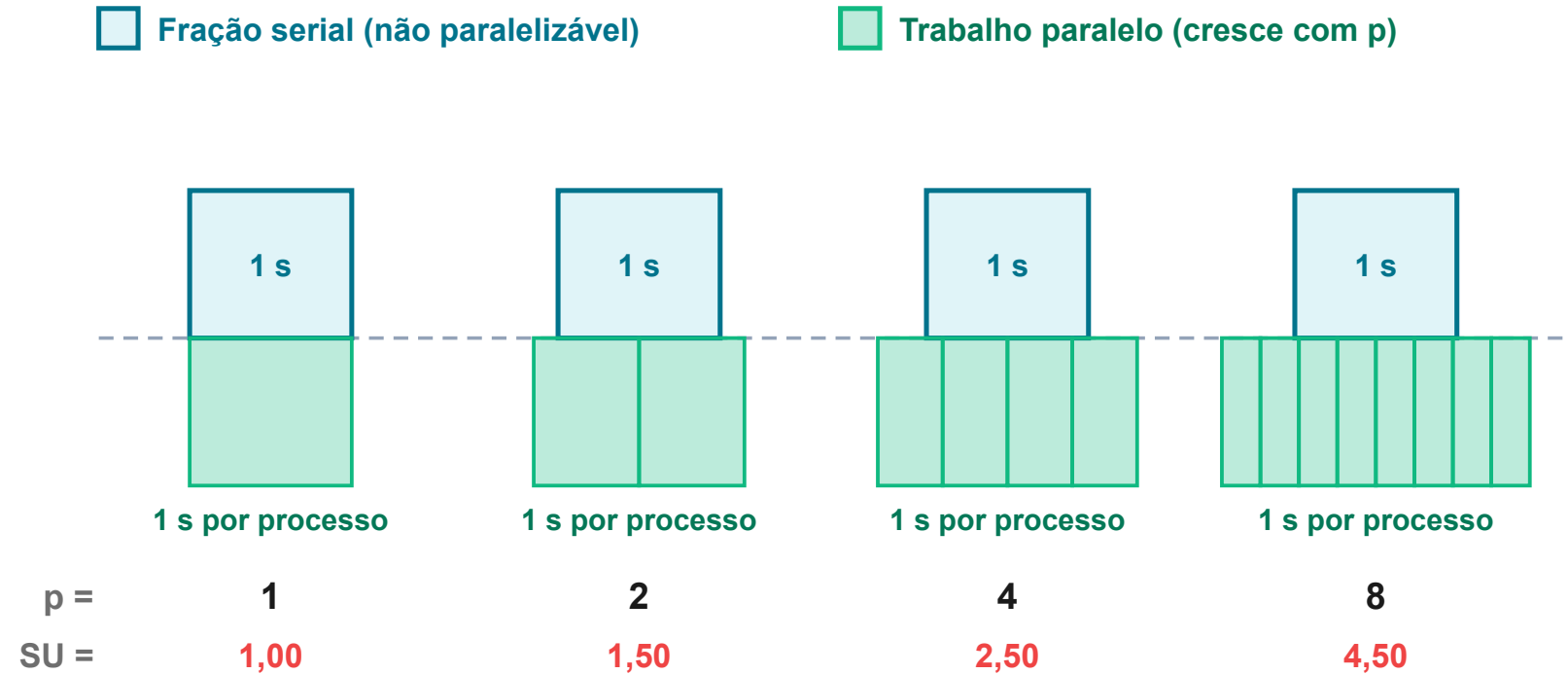
$$SU(N) = s + (p \times N)$$

Mesmas definições: s = fração sequencial, p = fração paralelizável, N = processadores

Mesmo exemplo: com $s = 0,1$ e 10 processadores, o ganho passa a ser de **9,1×**: $0,1 + (0,9 \times 10) = 9,1$

Amdahl pergunta *"quão rápido resolvo este problema?"*. Gustafson pergunta *"quão grande é o problema que consigo resolver neste tempo?"*

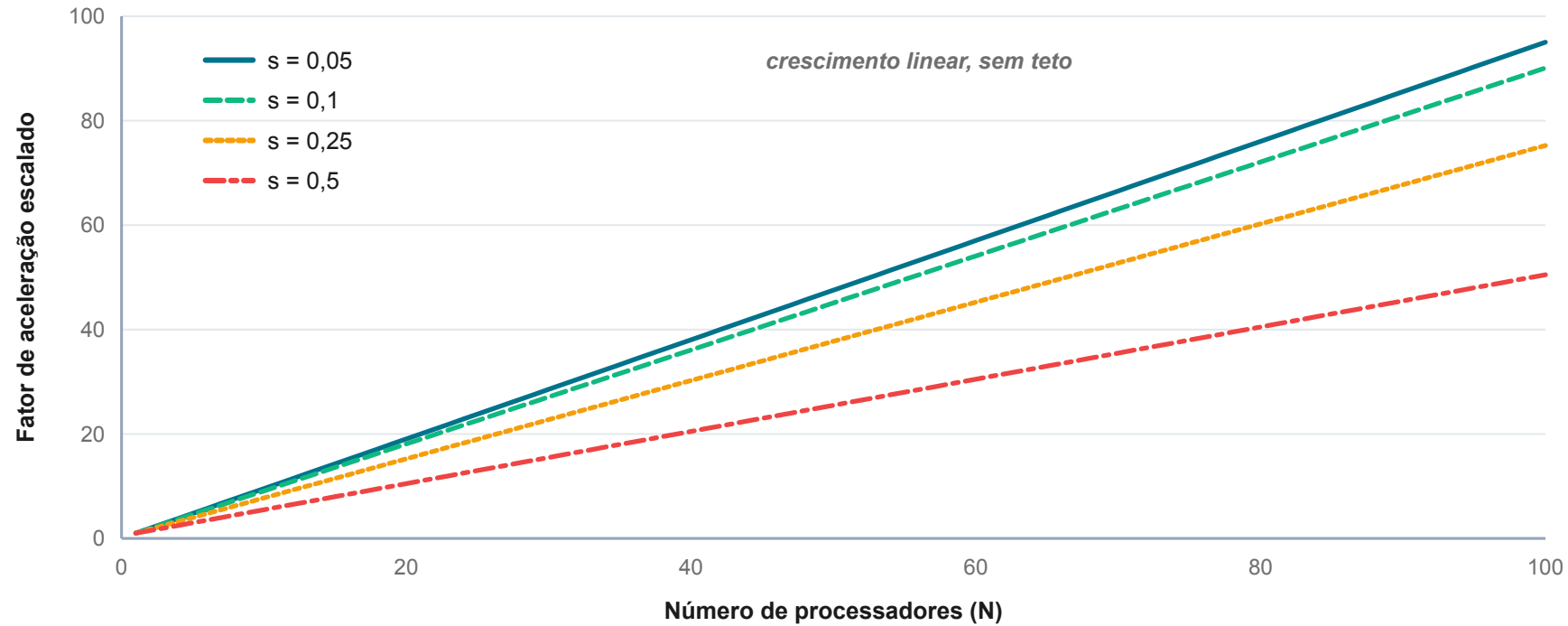
VISUALIZANDO A LEI DE GUSTAFSON



Problema que cresce com p: cada processo mantém 1 s de trabalho.

$$SU(8) = 9 / 2 = 4,50 \cdot \text{Eficiência} = 0,56$$

SEM TETO: O GANHHO CRESCE LINEARMENTE



A fração sequencial agora muda apenas a **inclinação** da reta, não impõe um limite.

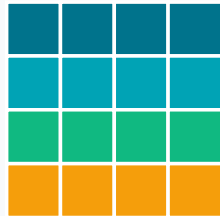
ESCALABILIDADE FORTE E FRACA

Escalabilidade forte

problema de tamanho FIXO



$p = 1$



$p = 4 \cdot$ cada um faz $\frac{1}{4}$

mesmo trabalho, menos tempo

Escalabilidade fraca

trabalho FIXO por processo



$p = 1$



$p = 4 \cdot$ problema $4\times$ maior

mais trabalho, mesmo tempo

COMPARAÇÃO ENTRE ESCALABILIDADE FORTE E FRACA

	Escalabilidade forte	Escalabilidade fraca
Tamanho do problema	Fixo	Cresce proporcionalmente a p
Trabalho por processo	Diminui com p	Constante
Lei associada	Amdahl	Gustafson
Eficiência	$E = SU/p = T_1 / (p \times T_p)$	$E = T_1 / T_p$ (sem $\div p$)
Comportamento ideal	Tempo cai pela metade ao dobrar p	Tempo permanece constante
Pergunta	Quão rápido resolvo <i>este</i> problema?	Que tamanho de problema cabe neste tempo?

As leis acima falam de **speed-up**; a medição é reportada em **eficiência**. Na forte, é a mesma $E = SU/p$ já vista. Na fraca, é uma definição **diferente**: a razão direta entre os tempos, porque o trabalho já cresceu junto com p .

COMO MEDIR NA PRÁTICA

ANTES DE MEDIR

- Compilar com otimização (`-O3`) nas duas versões
- Usar o **melhor** algoritmo sequencial como referência
- Fixar o tamanho do problema e a entrada
- Executar sozinho no nó: sem outros trabalhos competindo

AO MEDIR

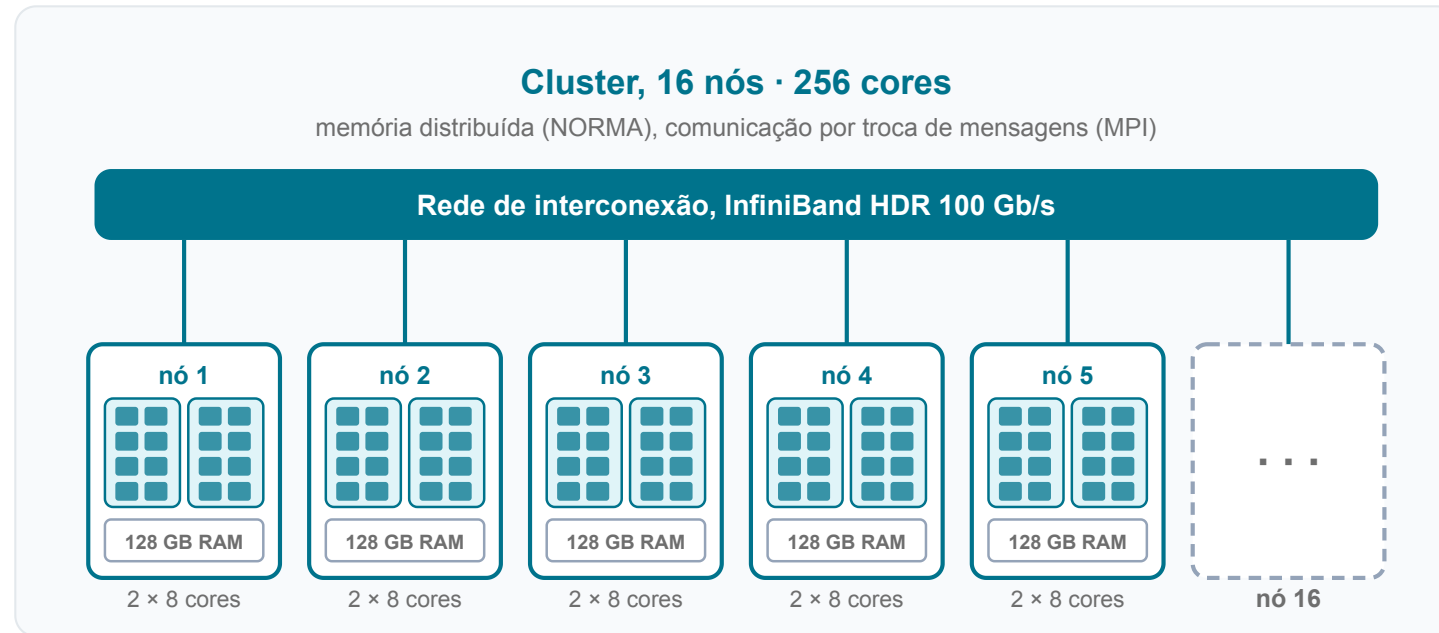
- Descartar a primeira execução (*warm-up* de cache)
- Repetir 5 a 10 vezes e reportar **mediana** e dispersão
- Variar p em potências de 2 até o limite da máquina
- Medir o tempo *interno* da região paralela, não só o total

Uma única execução por configuração não é medição: é anedota. A variabilidade entre execuções em cluster compartilhado chega facilmente a 10%.

ESTUDO DE CASO

Uma aplicação MPI em um cluster de 16 nós

A PLATAFORMA



**16 nós × 2 sockets × 8 cores =
256 cores**

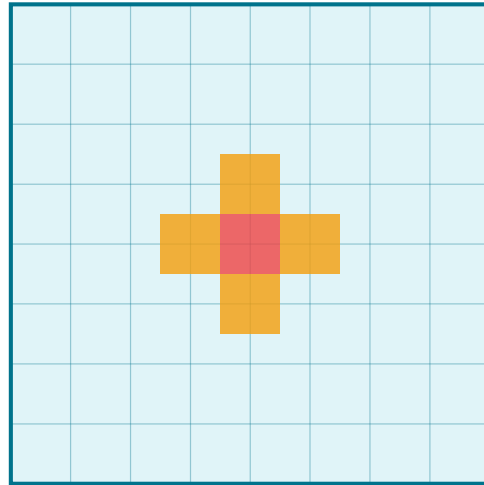
128 GB de RAM por nó (UMA
dentro do socket, NUMA entre
sockets)

InfiniBand HDR 100 Gb/s, latência
 $\approx 1,5 \mu\text{s}$

A APLICAÇÃO: DIFUSÃO DE CALOR 2D

Domínio $N \times N$

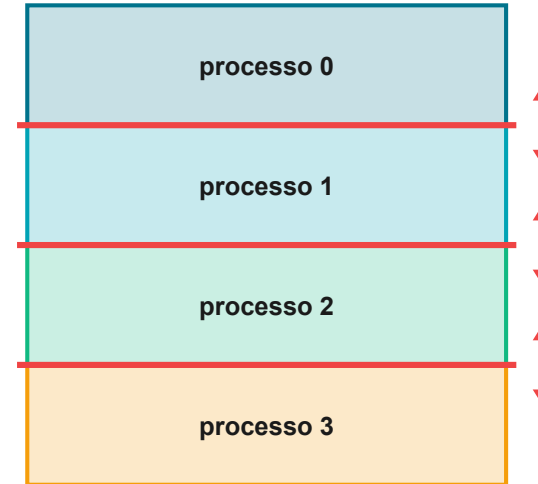
estêncil de 5 pontos



cada ponto depende dos 4 vizinhos

Decomposição 1D em faixas

N/p linhas por processo



troca de bordas (halo) a cada iteração

Método de **Jacobi** sobre malha de **8192 × 8192**, 5000 iterações, dupla precisão

MPI com decomposição 1D + `MPI_Allreduce` por iteração para testar convergência

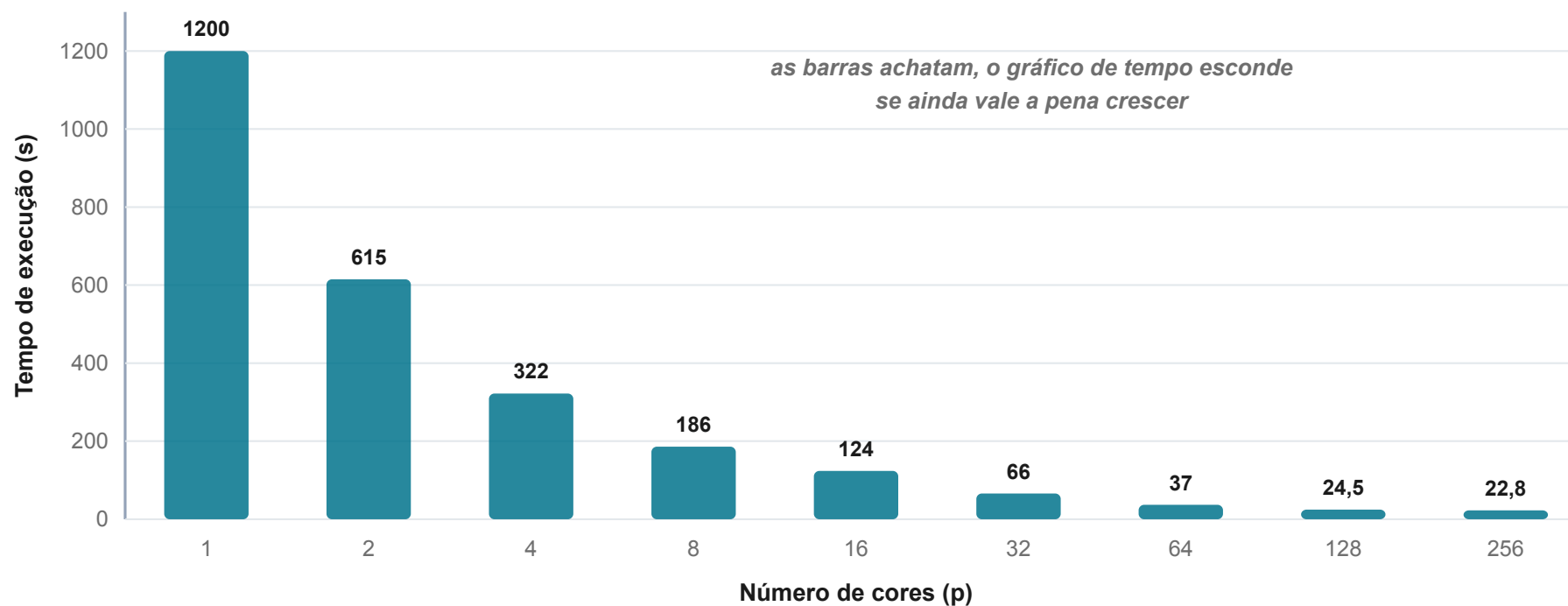
RESULTADOS: ESCALABILIDADE FORTE

Malha fixa em 8192 × 8192. Mediana de 10 execuções.

p (cores)	1	2	4	8	16	32	64	128	256
Nós	1	1	1	1	1	2	4	8	16
Tempo (s)	1200	615	322	186	124	66,0	37,0	24,5	22,8
Speed-up	1,00	1,95	3,73	6,45	9,68	18,2	32,4	49,0	52,6
Eficiência	1,00	0,98	0,93	0,81	0,60	0,57	0,51	0,38	0,21

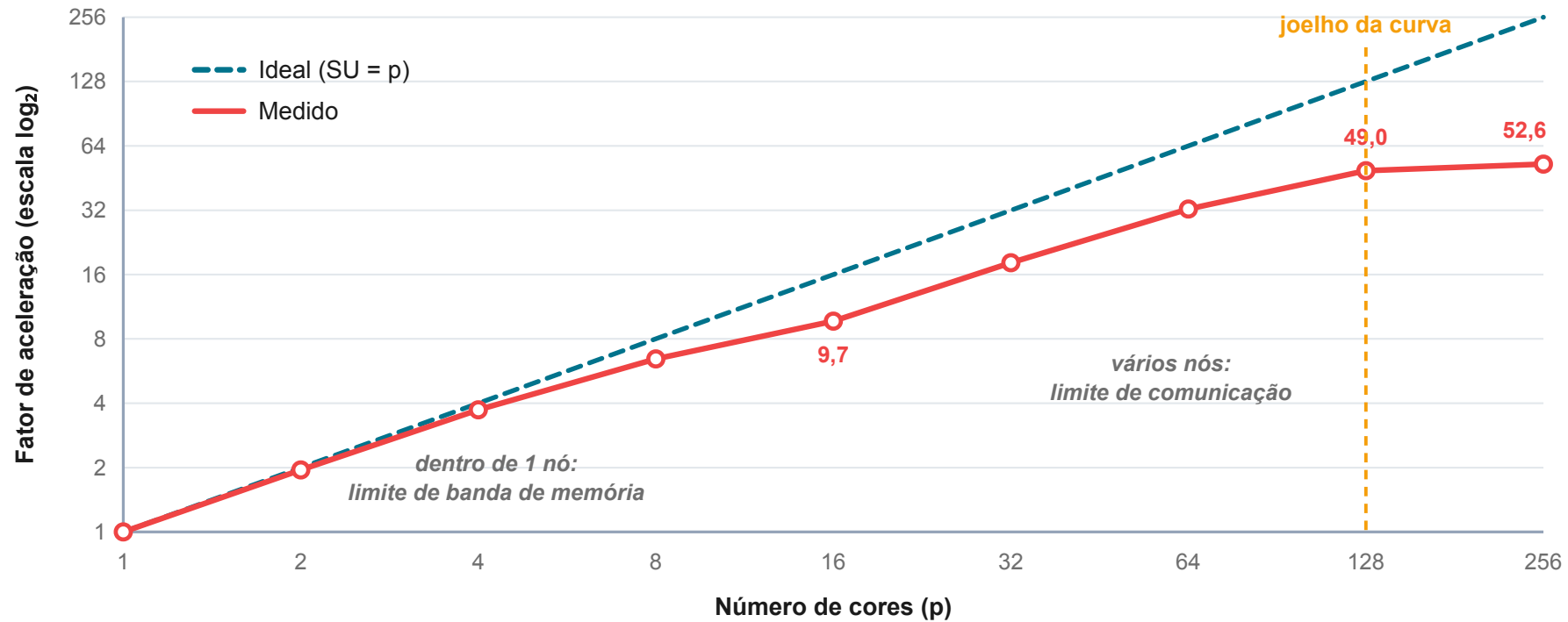
Dobrar de 128 para 256 cores economiza 1,7 s e derruba a eficiência de 0,38 para 0,21.

PLOTANDO O TEMPO



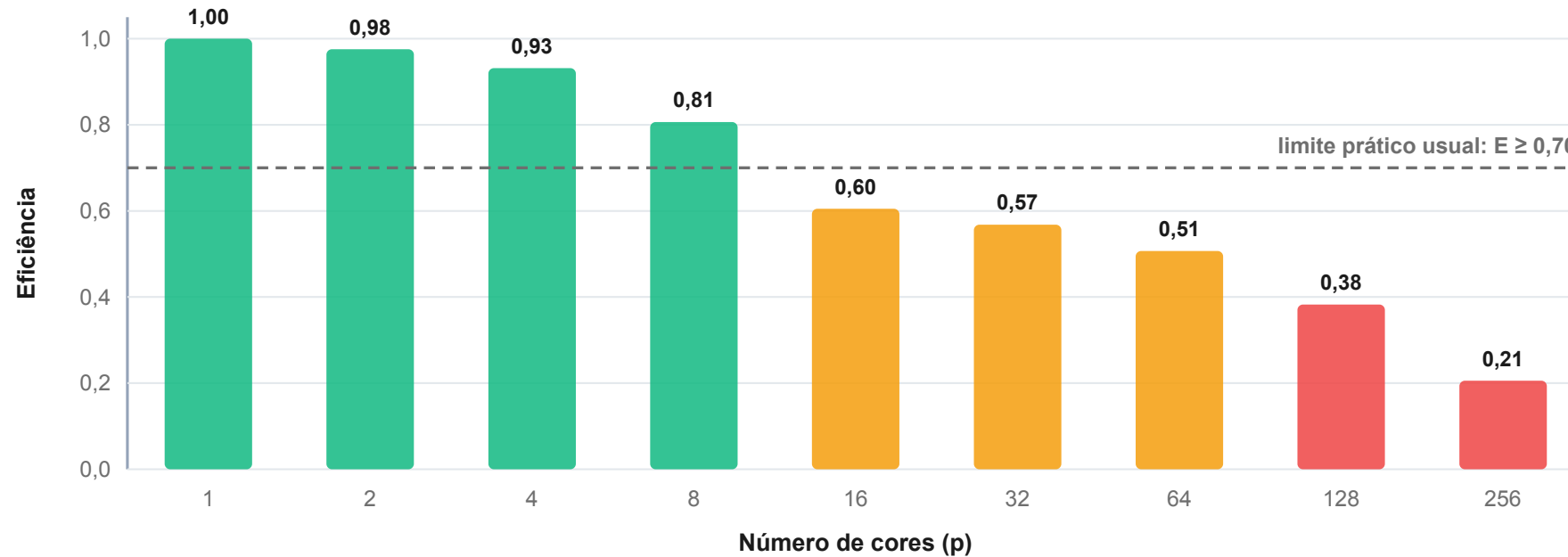
O gráfico de tempo responde *"quanto demorou?"*, mas esconde *"valeu a pena?"*: depois de 32 cores todas as barras parecem igualmente pequenas.

PLOTANDO O SPEED-UP



Em escala log-log, a aceleração ideal vira uma reta, e o afastamento da curva medida fica visível em toda a faixa.

PLOTANDO A EFICIÊNCIA



A queda mais brusca ocorre **dentro do nó**, de 8 para 16 cores: 0,81 → 0,60

De 16 a 64 cores a eficiência quase **estabiliza**: 0,60 → 0,51

COMPARANDO COM O LIMITE TEÓRICO

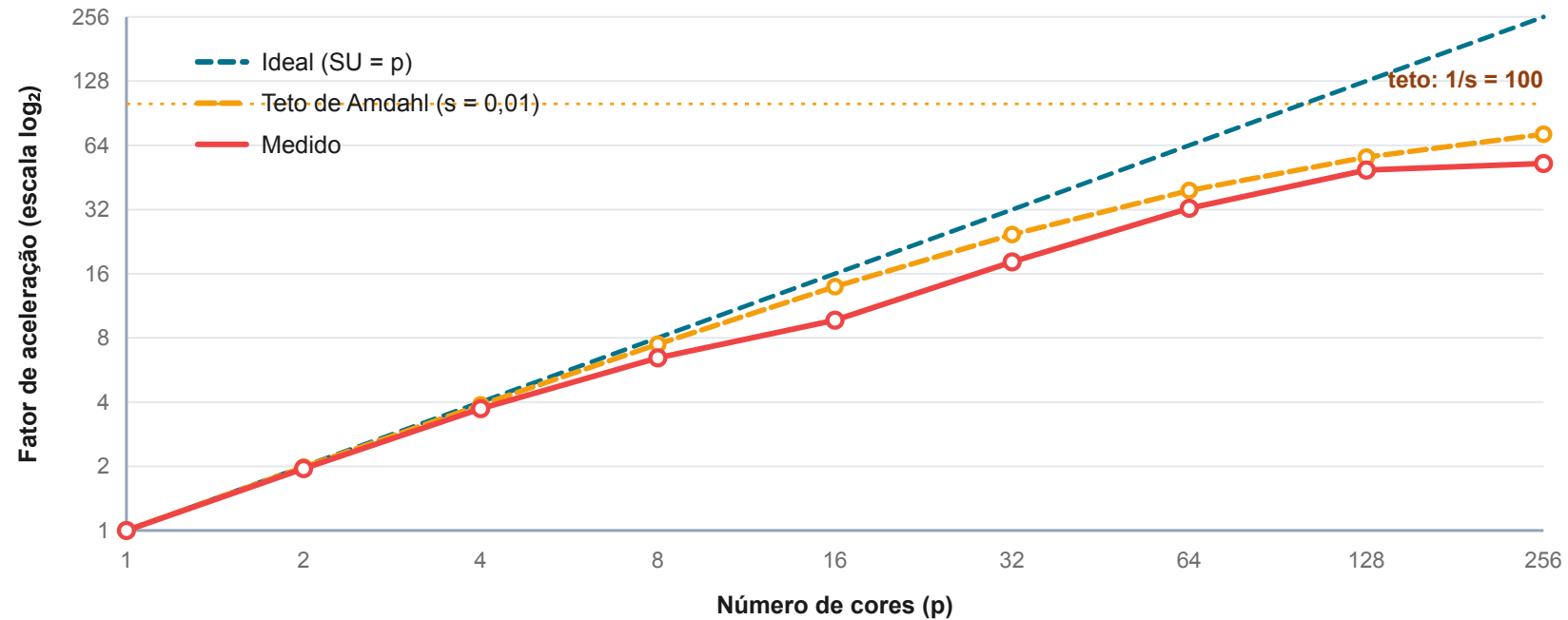
Fechando a **escalabilidade forte**: meça a fração sequencial por *profiling* e confronte o speed-up medido com o teto que ela impõe. Aqui, entrada, alocação e saída somam **12 s** dos 1200 s $\Rightarrow s = 0,01$.

p	16	64	128	256	$\rightarrow \infty$
Teto de Amdahl	13,9	39,3	56,4	72,1	100
Medido	9,7	32,4	49,0	52,6	n/a

Amdahl é um **teto**, não uma previsão: diz o que a fração sequencial permite, ignorando comunicação e sincronização. Sua curva medida fica sempre **abaixo** dele.

Na prática: calcule $1/s$ *antes* de pedir recursos. Com $s = 0,01$ o teto é 100×, nenhum esforço de otimização na parte paralela passa disso.

PLOTANDO O TETO SOBRE O MEDIDO



Entre a reta **ideal** e o **teto**: o que a fração sequencial já custa, mesmo sem nenhum overhead

Entre o **teto** e o **medido**: tudo o que Amdahl não modela, como comunicação, sincronização e memória

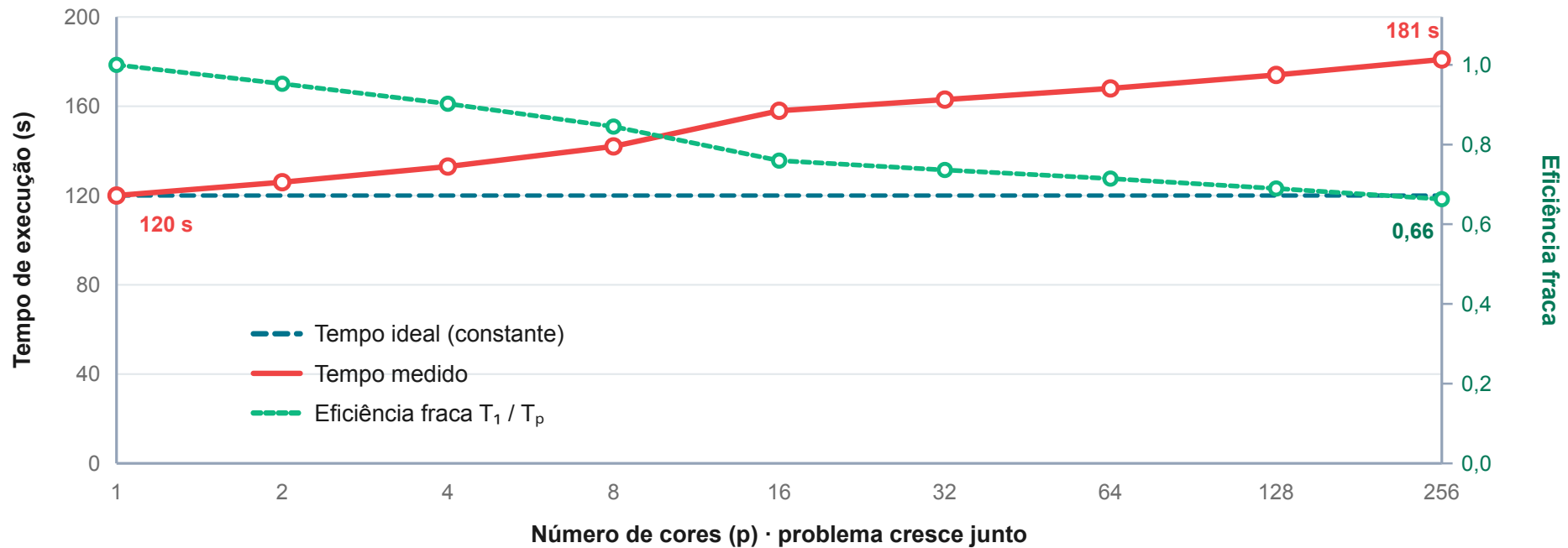
ESCALABILIDADE FRACA: O QUE MUDA NO EXPERIMENTO

Segundo experimento: mesmo código, mesmo cluster. O que muda é que o **tamanho do problema deixa de ser fixo**: ele cresce junto com p , para que cada processo receba sempre a **mesma carga**.

p (cores)	1	4	16	64	256
Malha global	2048 ²	4096 ²	8192 ²	16384 ²	32768 ²
Trabalho total	4,2 M	16,8 M	67,1 M	268 M	1074 M
Por processo	4,2 M	4,2 M	4,2 M	4,2 M	4,2 M

Na escalabilidade **forte** variávamos uma coisa só: p , com o problema parado. Aqui variam **duas ao mesmo tempo**: p e o tamanho do problema. Por isso o ideal deixa de ser "o tempo cai pela metade" e passa a ser "o tempo não muda".

PLOTANDO A ESCALABILIDADE FRACA



O ideal agora é a **horizontal**: mesmo tempo para um problema 256× maior

O problema cresce **256×** e o tempo sobe apenas **51%**.
A métrica é T_1/T_p , sem dividir por p

QUANTOS NÓS USAR?

RODADA DE PRODUÇÃO

Muitas execuções, orçamento de máquina limitado.

32 a 64 cores (2 a 4 nós),
eficiência $\approx 0,51$ a $0,57$

RESULTADO URGENTE

Uma execução, prazo apertado.

128 cores (8 nós), o joelho da curva, antes da estagnação

NUNCA

256 cores para esta malha:
dobra o custo para ganhar 7% de tempo.

Só se justifica com uma malha **maior**, como no teste de escalabilidade fraca.

A resposta não sai do código nem da máquina isoladamente: sai das **medições**, e muda com o tamanho do problema.

ARMADILHAS COMUNS NA MEDIÇÃO

ERROS DE REFERÊNCIA

- Usar o paralelo com $p = 1$ como T_1
- Comparar com um sequencial não otimizado
- Mudar o tamanho do problema entre execuções de um teste forte

ERROS DE EXECUÇÃO

- Medir uma única vez por configuração
- Compartilhar o nó com outros trabalhos
- Incluir tempo de E/S de disco na região cronometrada

Speed-up superlinear ($SU > p$) existe e tem causa legítima: com mais processos, cada subdomínio pode caber inteiro na cache. Mas quase sempre é sinal de erro na medição; desconfie e verifique antes de comemorar.

RESUMO

- **Speed-up** (T_1/T_p) diz se ficou mais rápido; **eficiência** (SU/p) diz a que preço
- Toda aplicação tem um **sweet spot**: mais recursos podem significar mais tempo
- **Amdahl** impõe o teto 1/s; **Gustafson** o remove quando o problema cresce
- Escalabilidade **forte** e **fraca** respondem perguntas diferentes
- A distância entre o previsto e o medido é o **custo dos complicadores**

Medir não é o fim do trabalho paralelo. É o que transforma "**funciona**" em "**vale a pena**".

"Seu programa escala? Como você sabe?"